

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics
COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Assessment Tool Weightage: 10%

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I **G. Ramya/192572073** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *G. Ramya*

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A recommendation engine processes user clicks, search queries, and purchases from multiple devices. Reverse engineer the probable data types, ingestion flow, and analytics stages.
2. A ride-sharing company displays live traffic heatmaps and demand forecasts. Reverse engineer the likely distributed storage and processing backbone behind the system.
3. A fraud detection dashboard flags suspicious card transactions in near real time. Reverse engineer the probable big data security, streaming, and alerting mechanisms used.
4. A social media platform stores videos, text comments, hashtags, and user engagement metrics. Reverse engineer how structured and unstructured data are probably managed together.
5. A manufacturing IoT platform collects machine logs and maintenance events from many plants. Reverse engineer the likely integration tools, storage strategy, and analytics path.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of	Good understanding	Partial understanding	Poor understanding

		underlying big data architecture			
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. Recommendation Engine

Probable data types:

- **Structured:** User ID, product ID, purchase amount, timestamp.
- **Semi-structured:** Clickstream events, search queries, device information in JSON.
- **Unstructured:** Search text, product descriptions, reviews.

Ingestion flow:

1. User clicks, searches, or purchases something.
2. Events are collected from websites, mobile apps, and other devices.
3. A streaming system such as **Apache Kafka** receives the events.
4. Data is stored in a distributed data lake or database.

Analytics stages:

- Clean and remove duplicate data.
- Combine activity from different devices using user IDs.
- Calculate user preferences and purchase patterns.
- Apply recommendation algorithms such as collaborative filtering or machine learning.
- Generate personalized product recommendations.

Design decision: A combination of **real-time streaming** + **batch processing** is likely used so that recommendations can be updated quickly while historical data is used for deeper analysis.

2. Ride-Sharing Traffic and Demand Forecasting

Probable backbone:

- GPS locations from vehicles and drivers.
- Trip requests, completed trips, cancellations, and timestamps.
- Traffic and map data from external sources.

Distributed storage:

- A distributed file system/data lake stores large historical datasets.
- Databases such as **HDFS**, **Apache Cassandra**, or **cloud object storage** can store large volumes of location and trip data.

Processing:

1. GPS and trip events are continuously ingested.
2. A streaming framework processes current traffic conditions.
3. Distributed processing frameworks such as **Apache Spark** analyze historical and current data.
4. Machine-learning models forecast future demand.
5. Results are sent to the application to display live heatmaps.

Design decision: Distributed processing is necessary because millions of GPS and trip events may arrive simultaneously from different locations.

3. Fraud Detection Dashboard**Probable data:**

- Card number/token.
- Transaction amount.
- Merchant information.
- Location and timestamp.
- Device and customer behavior.
- Previous transaction history.

Streaming mechanism:

1. Every card transaction generates an event.
2. Events are sent to a streaming platform such as **Kafka**.
3. Stream-processing tools analyze transactions immediately.
4. Rules and machine-learning models calculate a fraud risk score.

Security mechanisms:

- Encryption during data transfer and storage.
- Tokenization/masking of sensitive card information.
- Authentication and role-based access control.

- Audit logs to track access and activity.

Alerting:

- High-risk transactions are automatically flagged.
- Alerts are sent to the fraud-monitoring dashboard.
- Security teams can investigate suspicious transactions.

Design decision: Near-real-time stream processing is preferred instead of waiting for batch processing because fraudulent transactions need to be identified quickly.

4. Social Media Data Management

A social media platform handles **structured, semi-structured, and unstructured data** together.

Structured data:

- User IDs
- Post IDs
- Likes
- Shares
- Followers
- Timestamps

Semi-structured data:

- JSON user profiles.
- Hashtags and metadata.
- Engagement events.

Unstructured data:

- Videos
- Images
- Text comments
- Audio

Probable architecture:

1. User posts and interactions are collected through APIs and event streams.
2. Text and engagement data are stored in distributed databases.

3. Large videos and images are stored in object storage/data lakes.
4. Metadata is stored separately and linked using IDs.
5. Distributed processing analyzes trends, hashtags, recommendations, and user engagement.

Design decision: Different storage systems are used because videos require large object storage, while structured engagement data needs fast database access.

5. Manufacturing IoT Platform

Probable data sources:

- Machine sensors.
- Temperature, pressure, vibration, and speed readings.
- Machine logs.
- Maintenance records.
- Production information.

Integration tools:

- IoT gateways collect data from machines.
- **MQTT** or similar protocols transmit sensor data.
- **Apache Kafka** can ingest high-volume event streams.
- APIs can integrate maintenance and enterprise systems.

Storage strategy:

- Real-time sensor data → streaming/database system.
- Historical sensor data → data lake.
- Maintenance records → relational or NoSQL database.
- Large machine logs → distributed storage.

Analytics path:

1. Sensors generate data.
2. IoT gateways collect and transmit it.
3. Streaming platforms ingest the data.
4. Data is cleaned and transformed.
5. Real-time analytics detects abnormal machine behavior.

6. Historical data is analyzed using machine learning.
7. Predictive-maintenance models predict possible machine failures.
8. Alerts are sent to maintenance teams.

Design decision: The system combines **real-time analytics and historical batch analytics**. Real-time processing detects immediate problems, while historical data helps identify long-term patterns and predict failures.